

# Opérateurs binaires

---

# 1. Préambule



## 1.1 Rappels sur les propriétés d'une opération

Soit  $E$  un ensemble, et  $*$  une loi de composition interne sur  $E$ .

On rappelle que :

- $e \in E$  est **l'élément neutre** pour  $*$  veut dire que pour tout  $u \in E$ ,  $u * e = u$ .
- $*$  est **associative**, si pour tout  $u, v, w \in E$ ,  $(u * v) * w = u * (v * w)$ .
- $*$  est **commutative**, si pour tout  $u, v \in E$ ,  $u * v = v * u$ .
- $u \in E$  est **inversible** (ou symétrisable) par  $*$  s'il existe  $u^{-1} \in E$  tel que  $u * u^{-1} = e$ .

## 1.2 Décomposition en base $d$

Soit  $d \in \mathbb{N} - \{0, 1\}$ ,

### **Theorem 1.2.1 (Décomposition en base $d$ )**

Tout entier naturel  $n$  se décompose de façon **unique** en base  $d$ .

C'est à dire que pour tout  $n$ , qu'il existe une unique suite  $(u_n)_{n \in \mathbb{N}}$  à valeurs dans  $\llbracket 0, d - 1 \rrbracket$ , tel que :

$$n = \sum_{k=0}^{+\infty} u_k d^k$$

Par la suite, on désignera les nombres en base binaire de la façon suivante :  $010110_2$ .

## 2. Opérateurs

---

## 2.1 Opérateurs OU et ET logique

Soient  $a, b \in \mathbb{N}$ ,

### Definition 2.1.1 (OU logique ' $|$ ' )

L'opérateur ' $|$ ' réalise un **OU logique** entre  $a$  et  $b$ . C'est à dire que le résultat est l'application d'un OU entre chaque bit de  $a$  et  $b$ , du plus faible au plus fort.

### Definition 2.1.2 (ET logique '&' )

L'opérateur '&' réalise un **ET logique** entre  $a$  et  $b$ . C'est à dire que le résultat est l'application d'un ET entre chaque bit de  $a$  et  $b$ , du plus faible au plus fort.

*Exemple :*

Soient  $a = 011101_2$ , et  $b = 11010110_2$ ,

$a | b = 11011111_2$  et  $a \& b = 10100_2$

## 2.2 Opérateur NON logique et remarques

Soit  $a \in \mathbb{N}$ ,

### Definition 2.2.1 (NON logique ' $\sim$ ' )

L'opérateur ' $\sim$ ' réalise un **NON logique** sur chaque bit de  $a$ . C'est à dire qu'il inverse chaque bit de  $a$ .

*Exemple :*  $\sim 1010111_2 = 101000_2$

### ① Remarques

Les opérateurs  $\&$  et  $|$  sont **commutatifs** et **associatifs**. De plus ils sont **idempotants**, c'est à dire que pour tout  $u \in \mathbb{N}$ ,  $u | u = u$  et  $u \& u = u$ .

Les outils d'algèbre de Boole sont naturellement applicables, comme par exemple le théorème de Morgan.

## 2.3 Opérateur XOR

Soient  $a, b \in \mathbb{N}$ ,

### Definition 2.3.1 (XOR '^')

Comme & et |, l'opérateur '^' applique l'opération XOR entre chaque bit de  $a$  et  $b$ .

Pour rappel, voici la table de vérité de l'opérateur XOR :

| x | y | XOR |
|---|---|-----|
| 0 | 0 | 0   |
| 0 | 1 | 1   |
| 1 | 0 | 1   |
| 1 | 1 | 0   |

## 2.4 Remarques sur l'opérateur XOR

L'opérateur XOR est très utile et pratique à manier, car:

- il est **associative**, pour tout  $a, b, c \in \mathbb{N}$ ,  $(a \wedge b) \wedge c = a \wedge (b \wedge c)$ .
- il est **commutative**, pour tout  $a, b \in \mathbb{N}$ ,  $a \wedge b = b \wedge a$ .
- il admet un **élément neutre** qui est 0.
- tout élément  $a \in \mathbb{N}$  est **inversible** par XOR, et son inverse est  $a$ .

En d'autres termes, il est possible de créer une structure de groupe avec  $\mathbb{N}$  et XOR.

C'est précisément grâce à ces propriétés que nous utilisons l'opérateur pour disperser les valeurs dans un *set* ou *hashmap* (voir cours *Structures des données partie 2*), car en plus il n'est pas fastidieux de trouver l'inverse d'un élément.

## 2.5 Opérateurs *left shift* et *right shift*

Soient  $n, a \in \mathbb{N}$ ,

### Definition 2.5.1 (Right shift '>>')

$n \gg a$  veut dire que nous décalons les bits de  $a$  de  $n$  emplacements vers la droite. Les  $b$  bits qui dépassent vers la droite sont supprimés.

### Definition 2.5.2 (Left shift '<<')

Similairement,  $a \ll n$  veut dire que nous décalons les bits de  $a$  de  $n$  emplacements vers la gauche. Les nouveaux bits à droite de  $a$  sont égaux à 0.

*Remarques:*

- Un décalage de  $b$  bits vers la **droite** signifie que le nombre est divisé par  $2^b$ .
- Un décalage de  $b$  bits vers la **gauche** signifie que le nombre est multiplié par  $2^b$ .
- Selon si  $a$  a un nombre fixe de bits (ex: C++) ou non (ex: Python), un décalage vers la gauche peut écraser des bits.

### 3. En pratique

---

## 3.1 Problème des sommes de XOR

Nous allons travailler sur un exemple.

### Problème

Soit  $A$  une liste d'entiers, on note  $\mathcal{S}_A$  l'ensemble des sous-séquences de  $A$ , et  $\mu(B)$  le XOR de tout les éléments d'une liste d'entiers  $B$ .

On cherche à déterminer :

$$R = \sum_{s \in \mathcal{S}_A} \mu(s)$$

Nous fixerons par la suite :

- $n = |A|$

## 3.2 Analyse générale

On peut chercher à calculer le XOR de chaque sous-séquence de  $A$ , néanmoins cela n'est pas envisageable, car il existe  $2^n$  sous-séquences.

On remarque qu'on cherche à calculer une **somme** de XOR.

L'addition a des propriétés intéressantes pour simplifier le calcul, le problème c'est comment les exploiter ?

On sait aussi que la propriété la plus iconique du XOR, c'est que  $1 \oplus 1 = 0$ .

En d'autres termes, soit  $B$  une liste d'entiers :

### **Theorem 3.2.1**

Le  $i$ -ème bit de  $\mu(B)$  est activé, si et seulement si le nombre de valeurs dans  $B$  qui ont le  $i$ -ème bit activé est **impair**.

### 3.3 Implémentation

Nous allons utiliser la formule de **décomposition en base  $p$** .

On sait que pour tout  $s \in \mathcal{S}_A$ , le nombre de bits de  $s$  est majoré par le nombre de bits de  $\max(s)$  et on note  $M$  cette valeur.

Le but est de savoir le nombre de fois que le  $i$ -ème bit est activé parmi toutes les valeurs de  $\mu(s)$ , où  $s \in \mathcal{S}_A$ . On note  $c_i$  ce nombre, ainsi :

$$R = \sum_{s \in \mathcal{S}_A} \mu(s) = \sum_{i=1}^M c_i 2^i$$

Maintenant, il reste à calculer les valeurs de  $(c_i) \dots$

## 3.3 Implémentation

Nous allons donc définir  $(u_i)$  tel que pour tout  $i \in \mathbb{N}^*$ ,  $u_i$  est égal au nombre de valeurs dans  $A$  avec le  $i$ -ème bit activé.

Soit  $i \in \mathbb{N}^*$ ,

Il y a respectivement  $n - u_i$  valeurs qui n'ont pas le  $i$ -ème bit activé.

En utilisant le **théorème 3.2.1**, parmi les  $u_i$  valeurs restantes, on cherche à prendre un nombre impair de 1.

Ce qui donne :

$$c_i = (n - u_i) \times \sum_{k=1}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n}{2k+1} = 2^{n-1} \times (n - u_i)$$

Ainsi :

$$R = \sum_{i=1}^M c_i 2^i = \sum_{i=1}^M (n - u_i) \times 2^{n-1} 2^i = \sum_{i=1}^M (n - u_i) 2^{n-1+i}$$

Complexité de calcul :  $\mathcal{O}(n \log_2(\max(A))) = \mathcal{O}(nM)$

## 3.4 Anecdotes et *tips*

Il est assez courant dans les langages de programmation bas niveau, de manipuler les opérateurs binaires pour réduire l'usage mémoire.

Le plus classique est le système de *flag*, où chaque bit d'un nombre  $n$  indique si une option est activée.

Il reste ensuite à savoir si le  $i$ -ème bit est activé, il suffit de regarder si  $n \& (1 \ll i)$  est:

- positif (option activée)
- nul (option désactivée)

## 4. Crédits

---

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

## 5. Sources et ressources

---

- [CP algorithms](#)
- [USACO guide - Intro to bitwise operators](#)
- [Problème originel des sommes des xors des sous-séquence](#)